

Assignment 2 Design Document

Owen Muchini(A1877011) | COMP 1039 Design Patterns in C++

31 May 2026

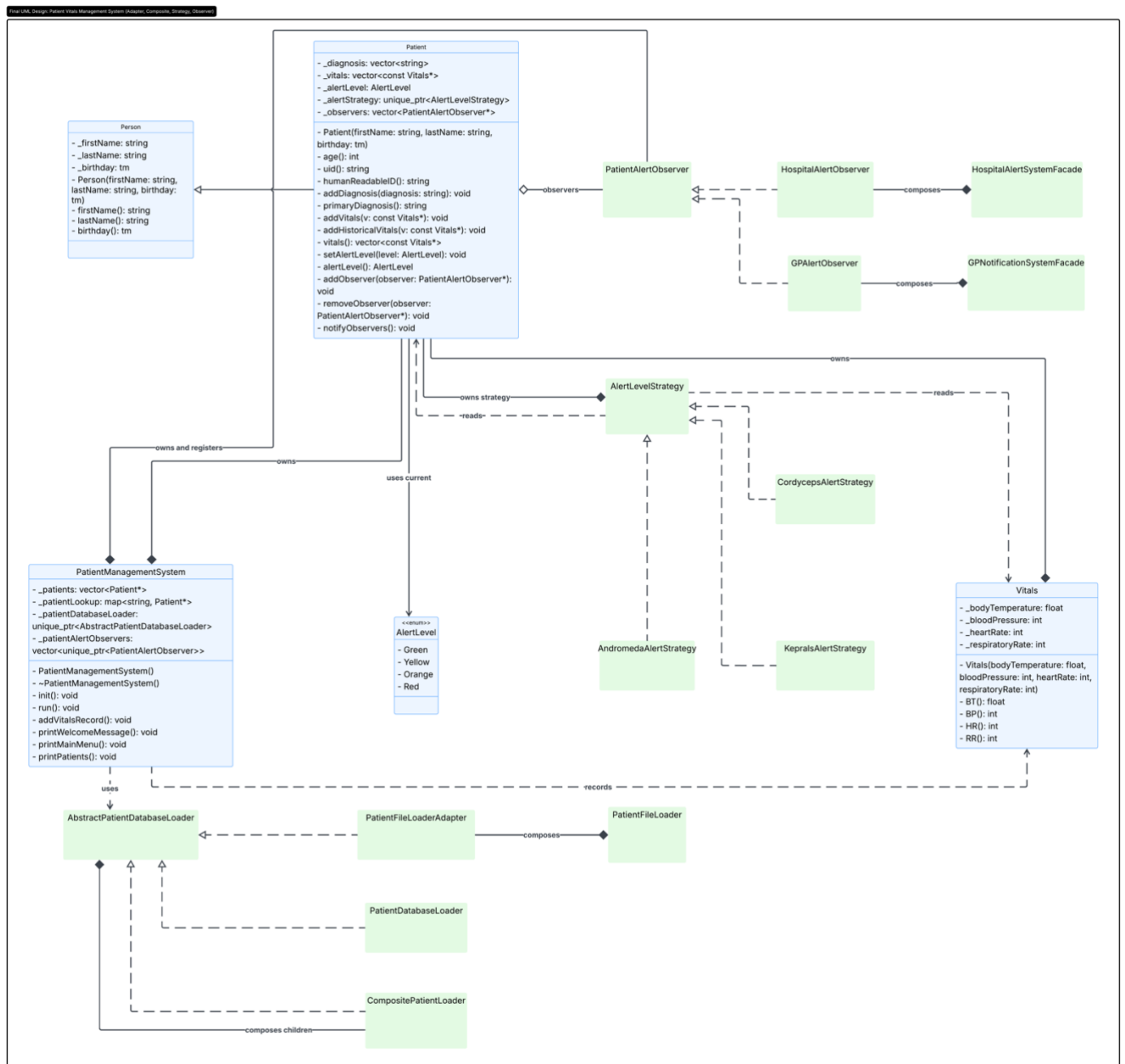
Table of Contents

Overview of System Design	3
FR1: Load Patients from File	4
FR2: Load Patients from File and Database	5
FR3: Calculate the Patient Alert Levels.....	6
FR4: Alert the Hospitals and GPs	7

Overview of System Design

This document presents the design of the Patient Vitals Management System. The system has been extended with four design patterns to address the functional requirements: the Adapter pattern (FR1), the Composite pattern (FR2), the Strategy pattern (FR3), and the Observer pattern (FR4).

The diagram below is the complete class design of the Patient Vitals Management System. Classes highlighted in green represent additions made to the original system design which implement the four design patterns. The specifics of each new class are described in detail within their corresponding functional requirement sections (FR1 through FR4).



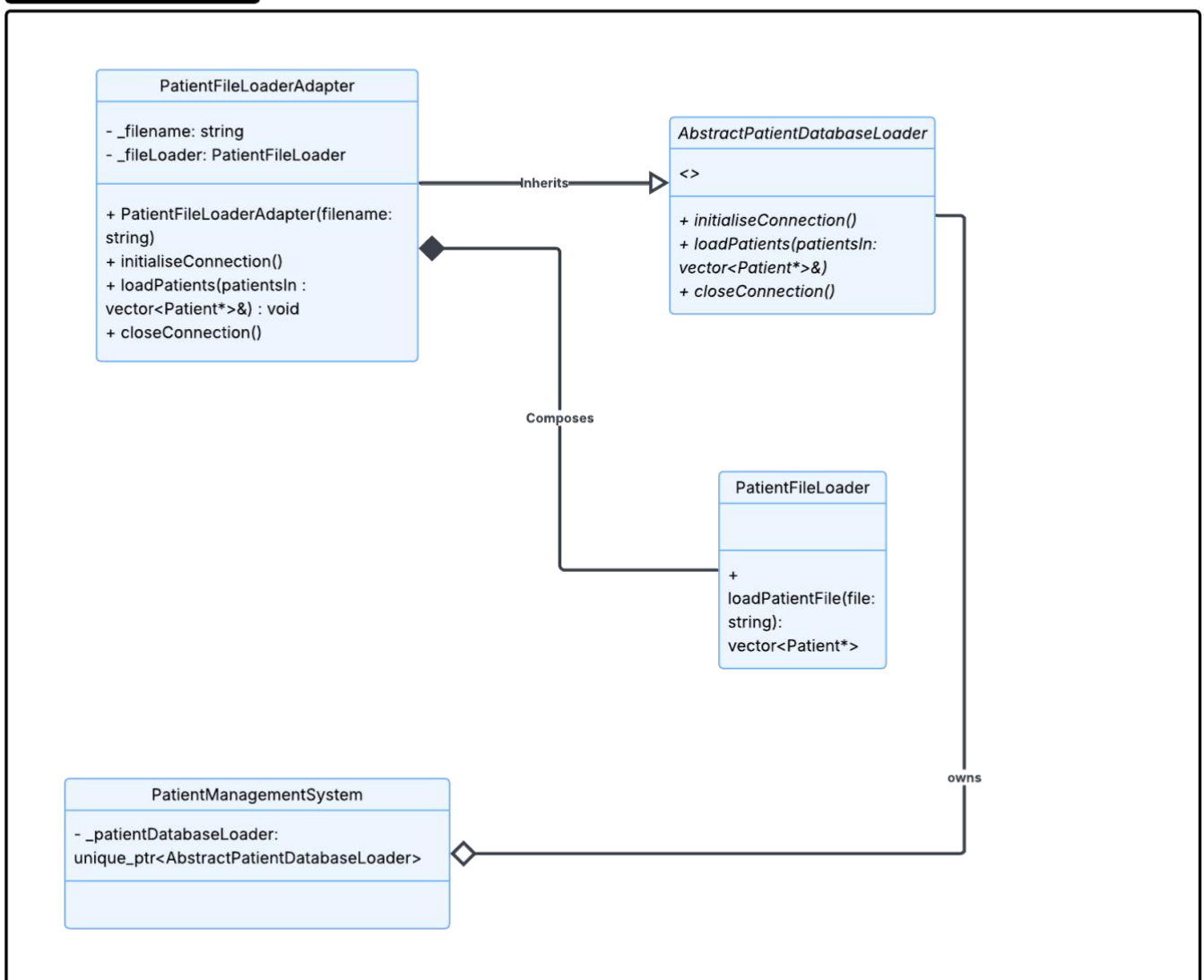
FR1: Load Patients from File

Besides needing to load the patient data from a file, the problem is that the existing system is already design to work with loaders through an existing “AbstractPatientDatabaseLoader” interface. The PatientManagementSystem (PMS) shouldn’t need to know if patients come from a database (DB) or from a text file. This change is necessary to prevent coupling the main system logic to a specific loading mechanism. The provided PatientFileLoader, does not match the interface expected by the system and exposes file-loading behaviour (through loadPatientFile()).

The Adapter pattern allows an existing class with an incompatible interface to be used when another interface is expected. In this design, AbstractPatientDatabaseLoader is the target interface, PatientFileLoader is the adaptee, and PatientFileLoaderAdapter is the adapter. The adapter inherits from AbstractPatientDatabaseLoader so it can be stored and used by PatientManagementSystem exactly like the original database loader.

FR1

Adapter Pattern: Patient Database Loader



FR1: How it works: Adapter Pattern

I added `PatientFileLoaderAdapter`. The adapter inherits from `AbstractPatientDatabaseLoader`, which means `PatientManagementSystem` can treat it the same way it treats the original database loader. Internally, the adapter stores the filename and contains a `PatientFileLoader` object. When the system calls `loadPatients()`, the adapter translates that call into a call to `PatientFileLoader::loadPatientFile()`. The patients returned by the file loader are then added into the patient vector passed in by the system.

Meaning the main patient management system doesn't need to know the details of how file loading works. It only knows that it has an object that behaves like an `AbstractPatientDatabaseLoader`. This makes sure that switching between the database loader and the file loader only requires changing the loader object created during initialisation.

I also added a small normalisation step for diagnosis names. The provided patient file included Kepral's Syndrome with a curly apostrophe, this displayed incorrectly on my Windows console as KepralİÇÖs Syndrome. This could also cause problems later when comparing diagnosis names in the alert strategy code. To avoid this, the file loader normalises curly apostrophes into a standard apostrophe before storing the diagnosis. This makes the loaded diagnosis consistently appear as Kepral's Syndrome and makes later string comparisons safer.

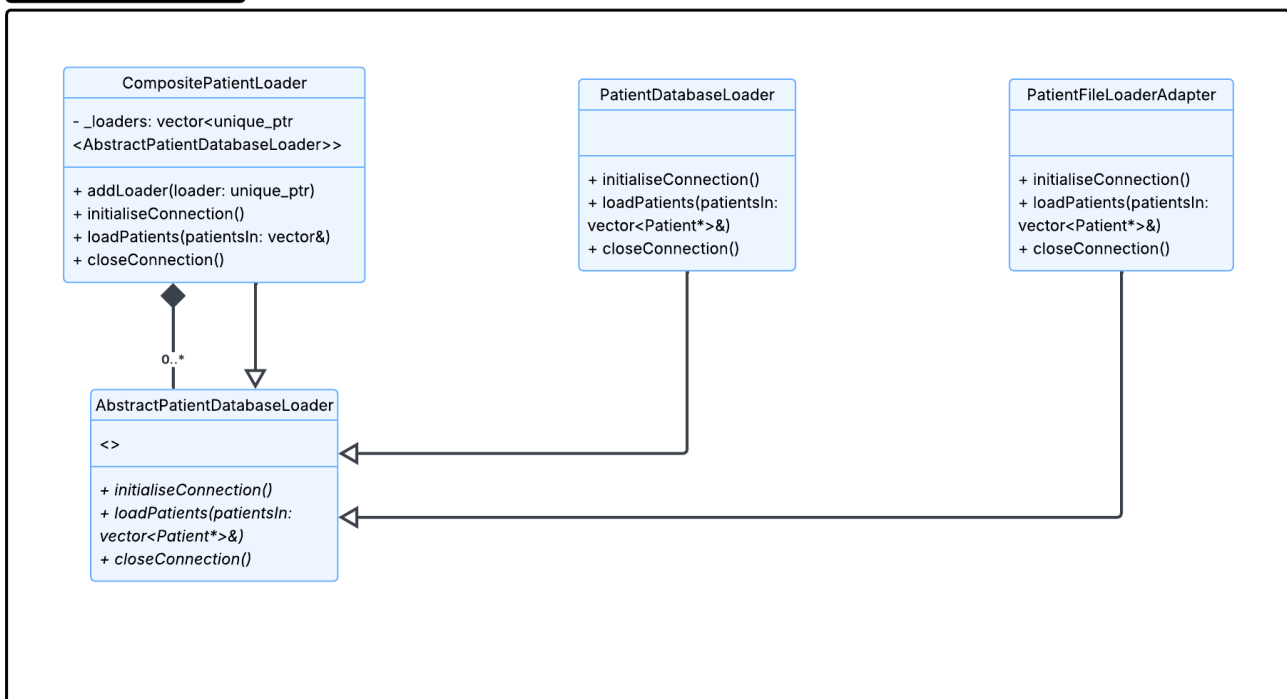
Git commits: I implemented the `PatientFileLoader` file parsing and added the `PatientFileLoaderAdapter` in commit 96eb13f. I fixed the apostrophe normalisation for diagnosis names in commit 664f765.

FR2: Load Patients from File and Database

The system must load patients from both the database and the text file simultaneously, while still making it trivial to switch between using the database loader alone, the file loader alone, or both together. The Composite pattern solves this by allowing individual loaders and groups of loaders to be treated uniformly through the same interface. I created a `CompositePatientLoader` class that inherits from `AbstractPatientDatabaseLoader` and internally holds a collection of `AbstractPatientDatabaseLoader` objects. When `loadPatients()` is called on the composite, it iterates through all its child loaders and delegates the call to each one, aggregating the results. Because the composite itself conforms to the same `AbstractPatientDatabaseLoader` interface, `PatientManagementSystem` can use it identically to any single loader. Switching between configurations (database only, file only, or both) requires changing only the loader setup in `PatientManagementSystem::init()`.

FR2

Composite Patient Loader UML Diagram



FR2: How the Composite design works

I added `CompositePatientLoader`. This class also inherits from `AbstractPatientDatabaseLoader`, so from the point of view of `PatientManagementSystem`, it still looks like a normal single patient loader. Internally it stores a collection of

other loaders. In the default setup, the composite contains a PatientDatabaseLoader first and a PatientFileLoaderAdapter second.

When PatientManagementSystem calls initialiseConnection(), loadPatients(), or closeConnection(), the composite loops through its child loaders and calls the same method on each one. For loading, this means the database loader adds its patients to the vector first, and then the file adapter adds the file patients afterwards. The result is that the system gets one combined patient list, while still only dealing with one abstract loader object.

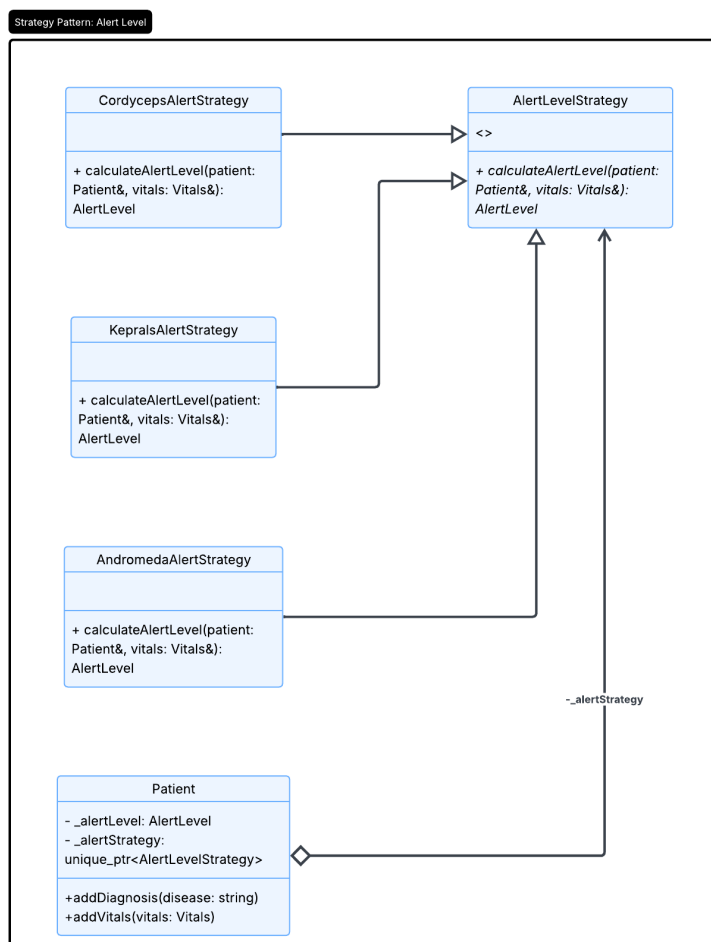
Git commits: I added CompositePatientLoader and adjusted the current composite loader in commit 5bd4022.

FR3: Calculate the Patient Alert Levels

FR3 requires the system to calculate a patient's alert level using different rules depending on the patient's disease. We need to decouple all the different calculations so it's not in one algorithm. Placing all of the rules directly inside Patient or PatientManagementSystem would create a long conditional structure that mixes patient data storage with disease-specific alert logic making the code harder to maintain

The Strategy pattern is appropriate because it encapsulates interchangeable algorithms behind a common interface. In this design, AlertLevelStrategy defines the common calculateAlertLevel() operation, while CordycepsAlertStrategy, KepralsAlertStrategy, and AndromedaAlertStrategy each implement one disease-specific algorithm. The Patient class acts as the context: it stores or selects the appropriate strategy and delegates alert calculation to it when new vitals are recorded. This keeps the alert rules separate from the patient's general responsibilities and makes the design easier to extend. If another disease is added later, the system can add another concrete strategy rather than rewriting the existing alert calculation logic.

FR3



FR3: How the Strategy design works

I added an abstract AlertLevelStrategy class with a calculateAlertLevel() method. Then I created one strategy class for each disease: CordycepsAlertStrategy, KepralsAlertStrategy, and AndromedaAlertStrategy. Each strategy only contains the rules for its own disease. So for example, the Cordyceps strategy checks respiratory rate thresholds, but the Andromeda strategy checks blood pressure thresholds.

The Patient class stores a unique_ptr to an AlertLevelStrategy. When a diagnosis is added to a patient, the patient chooses the matching strategy for that diagnosis. Later, when new vitals are recorded through addVitals(), the patient delegates the alert calculation to its strategy. The strategy returns the correct AlertLevel, and the patient updates its current alert level.

I also separated newly recorded vitals from historical vitals. I added addHistoricalVitals() for vitals loaded from existing data. This stores the vitals without triggering the strategy. The normal addVitals() method is used when the user records new vitals, and that is where the alert level calculation happens.

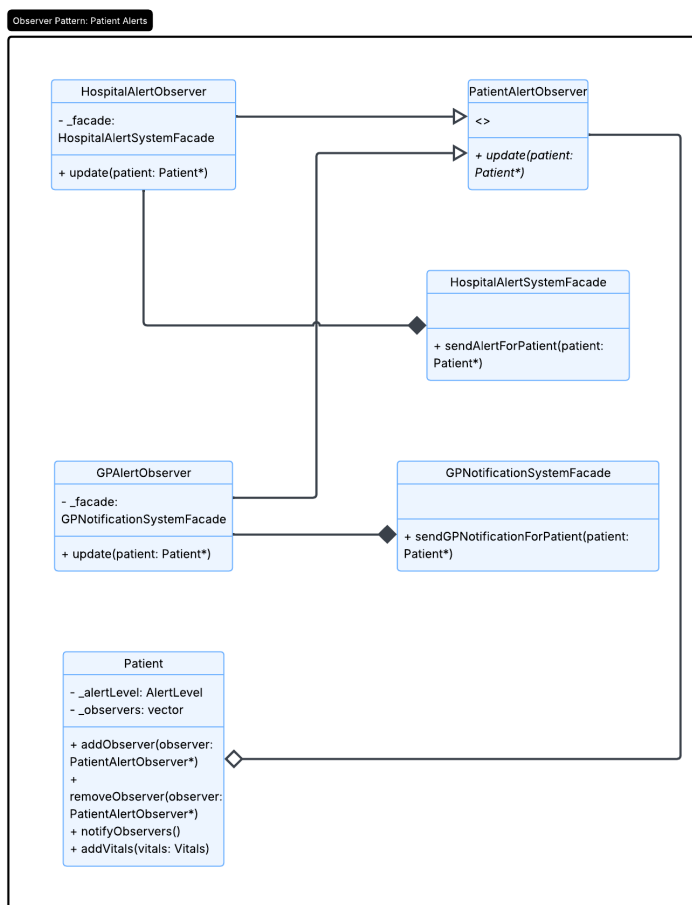
Git commits: I added AlertLevelStrategy, the three strategy classes, and updated Patient to use them in commit a15b76e.

FR4: Alert the Hospitals and GPs

FR4 requires hospitals and GPs to be notified immediately when a patient's alert level reaches Red. The design problem is that the patient alert calculation should not be tightly coupled to every external system that may need to react to an alert

The Observer pattern is appropriate because it allows interested objects to subscribe to changes in another object's state. In this design, Patient is the subject because its alert level is the state being monitored. PatientAlertObserver defines the observer interface, and the concrete observers adapt the hospital and GP notification facades into that interface. When a patient's alert level changes to Red, the patient notifies all registered observers. Each observer then delegates to its corresponding facade. This design keeps the patient alert mechanism decoupled from the specific notification systems, while still ensuring that notifications happen immediately when the relevant state change occurs.

FR4



FR4: How the Observer design works

I made Patient the subject in the Observer pattern. It stores a list of PatientAlertObserver pointers and provides methods to add, remove, and notify observers. I then added PatientAlertObserver as the abstract observer interface. The two observers are HospitalAlertObserver and GPAlertObserver.

The observers wrap the provided facade classes instead of modifying the facades directly. HospitalAlertObserver owns a HospitalAlertSystemFacade, and GPAlertObserver owns a GPNotificationSystemFacade. When an observer receives an update from a patient, it forwards the notification to the appropriate facade.

During system initialisation, PatientManagementSystem creates the hospital and GP observers and registers them with each loaded patient. Then, the Strategy pattern calculates the patient's new alert level. If the patient's alert level changes to Red, the patient calls notifyObservers(). This loops through the registered observers and calls update() on each one, causing both the hospital and GP notification systems to be triggered.

Git commits: I added PatientAlertObserver, HospitalAlertObserver, and GPAlertObserver, and worked on observer registration into PatientManagementSystem in commit a47708b.